

Guida Implementativa

SPM Modulo 1: Partition Mapping Kernel

Vectorizzazione del Kernel di Mapping Chiave→Partizione

Corso SPM — A.Y. 2025/26

Ultimo aggiornamento: March 19, 2026

Contents

1	Panoramica del Progetto	2
1.1	Obiettivo	2
1.2	Deliverables richiesti	2
1.3	Struttura del progetto	2
2	Fondamenti Teorici	2
2.1	Hashing: dal problema alla scelta	2
2.1.1	Hashing by Division ($k \bmod P$)	3
2.1.2	Universal Hashing (Ferragina, Cap. 8, §8.3)	3
2.1.3	Multiply-Add-Shift (Ferragina, Cap. 8, §8.3.1)	3
2.1.4	La nostra scelta: Fibonacci Hashing (Multiply-Shift)	3
2.2	SIMD e Vettorizzazione (Lezioni 7&8)	4
2.2.1	Auto-vettorizzazione del compilatore	4
2.2.2	Intrinsics AVX2 (Lezione 7&8)	4
2.3	Metriche di Performance (Lezione 10)	5
3	Workflow di Implementazione	5
3.1	Fase 1: Setup e Infrastruttura	5
3.2	Fase 2: Implementazione Plain C++ (Task 1)	6
3.2.1	Step 2.1: Scrivere il kernel	6
3.2.2	Step 2.2: Compilare i due binari	6
3.2.3	Step 2.3: Verificare la vettorizzazione	6
3.3	Fase 3: Implementazione AVX2 (Task 2)	7
3.3.1	Step 3.1: La decomposizione della moltiplicazione	7
3.3.2	Step 3.2: Il loop principale	7
3.3.3	Step 3.3: Packing del risultato	7
3.4	Fase 4: CUDA (Task 3 — Opzionale)	7
3.4.1	Step 4.1: Kernel CUDA	7
3.4.2	Step 4.2: Timing separati	7
3.4.3	Step 4.3: Compilazione	8
4	Testing e Validazione	8
4.1	Strategia di Correttezza	8
4.1.1	Livello 1: Checksum (per N grandi)	8
4.1.2	Livello 2: Confronto element-wise (per N piccolo)	8
4.1.3	Livello 3: Verifica integrata nell'AVX2	8
4.2	Strategia di Benchmark	8
4.3	Esperimenti suggeriti	9

4.4	Analisi della Distribuzione	9
5	Cluster: spmcluster (node09)	9
5.1	Architettura di node09	9
5.2	Comandi SLURM essenziali	9
5.3	Workflow di deployment	10
6	Analisi delle Performance e Bottleneck	10
6.1	Modello Roofline (Lezione 5&6)	10
6.2	Cosa limita le performance?	11
6.3	Cosa aspettarsi come speedup	11
7	Checklist per il Report Finale (max 4 pagine)	11
8	Consigli e Pitfall	12
9	Riferimenti	12

1 Panoramica del Progetto

1.1 Obiettivo

Implementare un kernel ad alte prestazioni che mappa un array di N chiavi `uint64_t` ad un array di partition identifier `part_id[i] ∈ [0, P)`, dove P è una potenza di 2. Questo è il primo stadio di un *partitioned hash join with duplicates*.

1.2 Deliverables richiesti

1. **Plain C++** (senza intrinsics): due binari dallo stesso sorgente:
 - **baseline**: compilato con auto-vectorization **disabilitata**
 - **autovec**: compilato con auto-vectorization **abilitata**
 - Evidenza della vettorizzazione tramite report GCC
2. **AVX2 intrinsics**: produce output identico alla versione plain
3. **(Opzionale) CUDA**: kernel-only time + transfer time separati; verifica correttezza vs CPU
4. **Report PDF** (max 4 pagine): scelte progettuali, evidence auto-vec, strategia intrinsics, risultati, analisi bottleneck

1.3 Struttura del progetto

```
module_1/
Makefile
README.md
include/common.hpp      # tipi, hash, keygen, timing, checksum
src/
  plain.cpp             # C++ plain → baseline + autovec
  avx2.cpp              # AVX2 intrinsics
  cuda_kernel.cu        # CUDA (opzionale)
scripts/
  run_bench.sh          # SLURM batch CPU benchmarks
  run_cuda.sh           # SLURM batch CUDA benchmarks
results/                # output benchmark + vec reports
report/                 # PDF report finale
guide/                  # questa guida
```

2 Fondamenti Teorici

2.1 Hashing: dal problema alla scelta

Il problema dell'hashing per il partitioning

Data una chiave $k \in U = \{0, \dots, 2^{64} - 1\}$ e un numero di partizioni P , vogliamo una funzione $h : U \rightarrow \{0, \dots, P - 1\}$ che sia:

- **Deterministica**: stessa chiave $\rightarrow$ stessa partizione
- **Uniformemente distribuita**: le chiavi si distribuiscano equamente tra le P partizioni
- **Veloce da calcolare**: il kernel viene applicato a *tutti* gli N input
- **SIMD-friendly**: componibile da operazioni vettorizzabili

2.1.1 Hashing by Division ($k \bmod P$)

Il metodo più semplice. **Problema:** l'operazione modulo è costosa (divisione intera), e se P è potenza di 2 si riduce a una maschera di bit ($k \& (P - 1)$), ma questo considera solo i bit **meno significativi** della chiave, ottenendo una distribuzione pessima per chiavi con pattern nei bit bassi.

2.1.2 Universal Hashing (Ferragina, Cap. 8, §8.3)

Da *Pearls of Algorithm Engineering* (Ferragina, 2023):

Una classe universale $\mathcal{H}$ di funzioni hash garantisce che, per qualsiasi coppia di chiavi distinte $x, y \in U$:

$$|\{h \in \mathcal{H} : h(x) = h(y)\}| \leq \frac{|\mathcal{H}|}{m}$$

Questo significa che una funzione scelta a caso da $\mathcal{H}$ ha probabilità $\leq 1/m$ di far collidere due chiavi fissate.

2.1.3 Multiply-Add-Shift (Ferragina, Cap. 8, §8.3.1)

Schema Multiply-Add-Shift (classe universale)

Assumendo $|U| = 2^w$ e table size $m = 2^\ell < |U|$:

$$h_{a,b}(k) = \left\lfloor \frac{(ak + b) \bmod 2^w}{2^{w-\ell}} \right\rfloor$$

dove a è un intero **dispari** $< |U|$ e $b \geq 0$.

Operativamente: seleziona gli ℓ bit “centrali” del prodotto $(ak + b)$.

Questa classe $\mathcal{H}_{w,\ell}$ è **universale** e può essere implementata **senza operazioni modulo** (solo moltiplicazioni e shift su registri di $2w$ bit).

2.1.4 La nostra scelta: Fibonacci Hashing (Multiply-Shift)

Semplifichiamo lo schema ponendo $b = 0$:

$$h(k) = (A \cdot k) \gg (64 - \log_2 P)$$

La costante A è scelta come:

$$A = \lfloor 2^{64}/\varphi \rfloor = \text{0x9E3779B97F4A7C15}$$

dove $\varphi = (1 + \sqrt{5})/2$ è il rapporto aureo (Knuth, TAOCP Vol. 3).

Suggerimento

Perché questa costante?

- È dispari per costruzione $\Rightarrow$ soddisfa il requisito della classe universale
- Il rapporto aureo ha proprietà di *mixing* ottimali: distribuisce le chiavi in modo massimamente uniforme evitando clustering
- I bit alti del prodotto $A \cdot k$ dipendono da **tutti** i bit di k , a differenza di $k \bmod P$ (solo bit bassi) o $k \gg s$ (solo bit alti)

Motivazioni per la scelta rispetto al progetto:

1. **Velocità:** una sola moltiplicazione + uno shift (nessuna divisione)
2. **SIMD-friendly:** la moltiplicazione si decompone in operazioni AVX2 a 32 bit; lo shift è uniforme su tutte le lane
3. **Distribuzione universale:** garanzia teorica di distribuzione $\leq 1/P$ di collisioni per coppia
4. **Nessuna tabella ausiliaria:** zero footprint di memoria aggiuntiva
5. **P potenza di 2:** lo shift è un parametro intero, nessun modulo

2.2 SIMD e Vettorizzazione (Lezioni 7&8)

Modello SIMD

Single Instruction, Multiple Data: la stessa istruzione è applicata simultaneamente a più dati in parallelo tramite unità vettoriali. Con AVX2:

- Registri a **256 bit** $\Rightarrow$ 4 interi a 64 bit, oppure 8 a 32 bit
- Speedup teorico: $\text{vector_width} \times \text{efficiency}$
- Nella pratica: 2–6 $\times$ per operazioni memory-bound

2.2.1 Auto-vettorizzazione del compilatore

Il compilatore (GCC) tenta di trasformare loop scalari in istruzioni SIMD. Condizioni necessarie (dalla lezione):

- Loop count noto all'ingresso
- Nessuna dipendenza tra iterazioni (no RAW/WAR/WAW)
- Nessun branch data-dependent nell'inner loop
- Nessuna function call non-inlinabile
- Accessi a memoria con stride unitario
- Assenza di aliasing (`__restrict__`)

Flag GCC per auto-vectorization:

```

1 # Abilitata (default con -O3):
2 g++ -O3 -march=native -ftree-vectorize
3
4 # Disabilitata (baseline):
5 g++ -O3 -march=native -fno-tree-vectorize
6
7 # Report:
8 -fopt-info-vec-optimized=report_ok.txt
9 -fopt-info-vec-missed=report_missed.txt

```

2.2.2 Intrinsics AVX2 (Lezione 7&8)

Problema: AVX2 **non ha** una moltiplicazione nativa 64-bit $\times$ 64-bit. `_mm256_mul_epu32` moltiplica i 32 bit bassi di ogni lane a 64 bit, producendo un risultato a 64 bit.

Strategia di decomposizione per calcolare $(A \cdot k) \bmod 2^{64}$:

$$\begin{aligned}
 A \cdot k &= A_{lo} \cdot k_{lo} && (64\text{-bit da mul_epu32}) \\
 &+ (A_{lo} \cdot k_{hi}) \ll 32 \\
 &+ (A_{hi} \cdot k_{lo}) \ll 32 \\
 &+ \underbrace{(A_{hi} \cdot k_{hi})}_{\text{overflow, ignorato}} \ll 64
 \end{aligned}$$

Dove $A_{lo} = A \bmod 2^{32}$, $A_{hi} = A \gg 32$, e analogamente per k .

Questo richiede **3 moltiplicazioni a 32 bit + 2 addizioni + 2 shift** per ogni gruppo di 4 chiavi.

2.3 Metriche di Performance (Lezione 10)

Metriche da riportare

- **Execution time** T (ms): mediana su r ripetizioni
- **Throughput** $= N/T$ (Mkeys/s): chiavi processate per secondo
- **Speedup** $S = T_{\text{baseline}}/T_{\text{ottimizzato}}$
- **Deviazione standard** σ su r ripetizioni

Roofline analysis (Lezione 5&6): il kernel è probabilmente **memory-bound**:

- Input: $N \times 8$ byte (chiavi 64-bit)
- Output: $N \times 4$ byte (partition id 32-bit)
- Compute: $\sim 3\text{--}5$ operazioni per chiave (mul + shift + store)
- Operational Intensity $\approx 5/12 \approx 0.4$ FLOP/byte $\Rightarrow$ fortemente memory-bound

Questo implica che il bottleneck sarà la **bandwidth di memoria**, non il compute.

3 Workflow di Implementazione

3.1 Fase 1: Setup e Infrastruttura

1. **Accesso al cluster**: connessione SSH a `spmcluster.unipi.it` (VPN se off-campus)
2. **Trasferimento codice**:

```
1 rsync -av module_1/ username@spmcluster.unipi.it:~/spm/module_1/
```

3. **Compilazione** su node09:

```
1 srun --partition=gpu-exclusive --nodelist=node09 --ntasks=1 bash
2 cd ~/spm/module_1
3 make all
```

4. **Test correttezza** (piccolo N):

```
1 make test_correctness
```

Attenzione

rsync: non copiare mai direttamente nella home (`~/`) con `-a`, rischi di alterare i permessi. Usa sempre una sottodirectory (es. `~/spm/`).

3.2 Fase 2: Implementazione Plain C++ (Task 1)

3.2.1 Step 2.1: Scrivere il kernel

Il file `src/plain.cpp` contiene il kernel:

```
1 void partition_map(const key_t* __restrict__ keys,
2                   part_t* __restrict__ part_ids,
3                   size_t N, unsigned shift) {
4     for (size_t i = 0; i < N; i++) {
5         part_ids[i] = static_cast<part_t>((HASH_A * keys[i]) >> shift);
6     }
7 }
```

Accorgimenti per favorire l'auto-vectorization:

- `__restrict__`: elimina il rischio di aliasing percepito dal compilatore
- Loop semplice con stride unitario e conteggio noto
- Nessuna function call nel body (tutto inline)
- Nessuna dipendenza tra iterazioni
- Memoria allineata a 32 byte (per load/store efficienti)

3.2.2 Step 2.2: Compilare i due binari

```
1 # Baseline (no auto-vec)
2 g++ -std=c++20 -O3 -march=native -mavx2 -fno-tree-vectorize \
3     src/plain.cpp -I include -o bin/plain_baseline
4
5 # Autovec (con report)
6 g++ -std=c++20 -O3 -march=native -mavx2 -ftree-vectorize \
7     -fopt-info-vec-optimized=results/vec_optimized.txt \
8     -fopt-info-vec-missed=results/vec_missed.txt \
9     src/plain.cpp -I include -o bin/plain_autovec
```

3.2.3 Step 2.3: Verificare la vettorizzazione

```
1 cat results/vec_optimized.txt
2 # Aspettati qualcosa come:
3 # src/plain.cpp:35:5: optimized: loop vectorized using 32 byte vectors
```

Attenzione

Se il report dice “not vectorized: ...”, analizza il motivo:

- “unsupported data type”: la moltiplicazione 64-bit potrebbe non vettorizzarsi. Se accade, il compilatore potrebbe comunque vettorizzare parzialmente.
- “possible aliasing”: aggiungi `__restrict__`
- “loop carried dependence”: verifica che non ci siano dipendenze

Se GCC non riesce a vettorizzare il loop 64-bit multiply, **documentalo nel report**: è un risultato significativo che motiva l'uso degli intrinsics.

3.3 Fase 3: Implementazione AVX2 (Task 2)

3.3.1 Step 3.1: La decomposizione della moltiplicazione

Il cuore dell'implementazione AVX2 è la funzione `mul64_avx2`:

```

1 static inline __m256i mul64_avx2(__m256i a, __m256i k) {
2     __m256i lo_lo = _mm256_mul_epu32(a, k); // a_lo * k_lo
3     __m256i a_hi = _mm256_srli_epi64(a, 32);
4     __m256i k_hi = _mm256_srli_epi64(k, 32);
5     __m256i cross = _mm256_add_epi64(
6         _mm256_mul_epu32(a, k_hi), // a_lo * k_hi
7         _mm256_mul_epu32(a_hi, k)); // a_hi * k_lo
8     cross = _mm256_slli_epi64(cross, 32);
9     return _mm256_add_epi64(lo_lo, cross);
10 }
```

3.3.2 Step 3.2: Il loop principale

```

1 for (size_t i = 0; i < simd_end; i += 4) {
2     __m256i vk = _mm256_load_si256(&keys[i]); // 4 chiavi
3     __m256i prod = mul64_avx2(va, vk); // 4 prodotti
4     __m256i vhash = _mm256_srli_epi64(prod, shift); // 4 shift
5     // pack 4x64-bit -> 4x32-bit e store
6     ...
7 }
8 // Tail: loop scalare per i rimanenti N%4 elementi
```

3.3.3 Step 3.3: Packing del risultato

I partition id sono nei 32 bit bassi di ogni lane a 64 bit. Per scriverli come 4 `uint32_t` consecutivi, usiamo `_mm256_permutevar8x32_epi32` con indici `{0,2,4,6,...}` e poi `_mm_storeu_si128` dei 128 bit bassi.

Suggerimento

Verifica: dopo ogni modifica all'implementazione AVX2, esegui il test di correttezza per controllare che il checksum corrisponda al riferimento scalare.

3.4 Fase 4: CUDA (Task 3 — Opzionale)

3.4.1 Step 4.1: Kernel CUDA

```

1 __global__ void partition_map_cuda(const uint64_t* keys,
2                                   uint32_t* part_ids,
3                                   size_t N, uint64_t hash_a,
4                                   unsigned shift) {
5     size_t idx = blockIdx.x * blockDim.x + threadIdx.x;
6     if (idx < N)
7         part_ids[idx] = (uint32_t)((hash_a * keys[idx]) >> shift);
8 }
```

3.4.2 Step 4.2: Timing separati

Il progetto richiede esplicitamente di riportare **separatamente**:

- **H→D transfer:** `cudaMemcpy(HostToDevice)` delle chiavi

- **Kernel execution:** solo il tempo del kernel GPU
- **D→H transfer:** cudaMemcpy(DeviceToHost) dei risultati

Usare **CUDA Events** per misurazioni accurate.

Suggerimento

Usa **pinned memory** (cudaMallocHost) per l'host buffer: migliora il throughput dei trasferimenti PCIe significativamente.

3.4.3 Step 4.3: Compilazione

```
1 # node09 ha una NVIDIA A30 (compute capability 8.0)
2 nvcc -std=c++17 -O3 -I include \
3     -gencode arch=compute_80,code=sm_80 \
4     src/cuda_kernel.cu -o bin/cuda_kernel
```

4 Testing e Validazione

4.1 Strategia di Correttezza

4.1.1 Livello 1: Checksum (per N grandi)

Ogni implementazione calcola un hash FNV-1a dell'array di output:

```
1 uint64_t h = 0xCBF29CE484222325ULL; // offset basis
2 for (size_t i = 0; i < N; i++) {
3     h ^= part_ids[i];
4     h *= 0x100000001B3ULL; // FNV prime
5 }
```

Se i checksum di due implementazioni coincidono $\Rightarrow$ gli output sono identici (con probabilità $> 1 - 2^{-64}$).

4.1.2 Livello 2: Confronto element-wise (per N piccolo)

Per $N \leq 32$, tutti i binari stampano l'output completo. Verifica manualmente o con **diff**:

```
1 bin/plain_baseline 16 4 42 > /tmp/ref.txt
2 bin/avx2 16 4 42 > /tmp/avx.txt
3 diff /tmp/ref.txt /tmp/avx.txt
```

4.1.3 Livello 3: Verifica integrata nell'AVX2

Il binario bin/avx2 esegue **internamente** sia il kernel scalare che quello AVX2, confronta i checksum, e riporta PASS/FAIL. In caso di FAIL, indica la prima posizione discordante.

4.2 Strategia di Benchmark

Attenzione

Metodologia richiesta dal progetto: multiple ripetizioni, reporting di mediana e deviazione standard.

1. **Ripetizioni:** 11 (dispari $\rightarrow$ mediana senza ambiguità)

2. **Warmup:** 1 esecuzione prima delle misurazioni (popola cache, TLB)
3. **Metrica:** `std::chrono::high_resolution_clock`
4. **Report:** mediana e σ dei tempi
5. **Throughput:** N/T_{median} in Mkeys/s

4.3 Esperimenti suggeriti

Table 1: Piano sperimentale

Esperimento	Variabile	Fisso	Obiettivo
Sweep N	$N \in \{1\text{M}, 10\text{M}, 50\text{M}, 100\text{M}, 200\text{M}\}$	$P = 256$	Stabilità del throughput; effetti
Sweep P	$P \in \{2, 4, 8, \dots, 1024\}$	$N = 100\text{M}$	Impatto di P su performance e c
Sweep <code>key_space</code>	$\{10^3, 10^5, 10^7, 10^9, 0\}$	$N = 100\text{M}, P = 256$	Sensibilità alla distribuzione in p

4.4 Analisi della Distribuzione

Per verificare che la funzione hash distribuisca uniformemente:

```

1 # Il programma stampa automaticamente min/max/expected per P<=1024
2 bin/avx2 100000000 256
3 # Output:
4 #   Distribution: min=389500 max=391200 expected=390625.0
5 #               ratio_max/exp=1.0015

```

Un `ratio_max/exp` vicino a 1.0 indica distribuzione uniforme.

5 Cluster: spmcluster (node09)

5.1 Architettura di node09

- **CPU:** AMD-based (Zen), supporto AVX2
- **GPU:** NVIDIA A30 (compute capability 8.0)
- **Partizioni SLURM:** `gpu-shared`, `gpu-exclusive`
- **Time limit:** 30 minuti per job

5.2 Comandi SLURM essenziali

```

1 # Informazioni sulle partizioni
2 sinfo
3
4 # Sessione interattiva su node09
5 salloc --partition=gpu-exclusive --odelist=node09 \
6       --ntasks=1 --cpus-per-task=4 --time=00:15:00
7
8 # Sottomissione batch
9 sbatch scripts/run_bench.sh
10
11 # Stato dei job
12 squeue --me
13

```

```

14 # Cancellare un job
15 scancel <jobid>

```

Attenzione

Non eseguire benchmark sul frontend! Il frontend è condiviso e i risultati non sarebbero affidabili. Usa sempre SLURM per allocare node09.

5.3 Workflow di deployment

```

1 # 1. Sul tuo Mac, trasferisci il codice
2 rsync -av module_1/ user@spmcluster.unipi.it:~/spm/module_1/
3
4 # 2. SSH al cluster
5 ssh user@spmcluster.unipi.it
6
7 # 3. Compila (puo' essere fatto sul frontend per i target CPU)
8 cd ~/spm/module_1
9 make all
10
11 # 4. Sottometti il benchmark
12 sbatch scripts/run_bench.sh
13
14 # 5. Monitora
15 squeue --me
16 # Quando completato:
17 cat results/bench_*.out
18
19 # 6. Trasferisci i risultati indietro
20 # (dal Mac)
21 rsync -av user@spmcluster.unipi.it:~/spm/module_1/results/ \
22     module_1/results/

```

6 Analisi delle Performance e Bottleneck

6.1 Modello Roofline (Lezione 5&6)

Il kernel è un **streaming kernel**: legge $N \times 8$ byte (chiavi), scrive $N \times 4$ byte (partizioni) = $12N$ byte di traffico memoria. Il compute è ~ 5 operazioni intere per chiave.

Operational Intensity (OI):

$$OI = \frac{\text{operazioni}}{\text{byte trasferiti}} = \frac{5N}{12N} \approx 0.42 \text{ op/byte}$$

Con bandwidth memoria di ~ 50 GB/s (tipica per un nodo), il tetto di memoria implica:

$$\text{Performance}_{\max} \approx 50 \times 10^9 / 12 \approx 4.2 \text{ Gkeys/s}$$

Suggerimento

Nel report, confronta il throughput misurato con questo upper bound teorico. Se il throughput è vicino al limite di bandwidth, il kernel è **memory-bound** e ulteriori ottimizzazioni compute non aiuteranno significativamente.

6.2 Cosa limita le performance?

1. **Memory bandwidth:** fattore dominante per questo kernel. L'AVX2 non può superare la banda di memoria.
2. **Compute:** la decomposizione della moltiplicazione 64-bit richiede 3 `mul_epu32` + addizioni + shift. Più costoso di un semplice add, ma comunque dominato dalla memoria per N grandi.
3. **Overhead SIMD:** packing dei risultati 64-bit $\rightarrow$ 32-bit, tail loop per $N\%4$.
4. **Per CUDA:** il trasferimento PCIe Host $\leftrightarrow$ Device domina per questo kernel semplice. Il kernel in sé è molto veloce.

6.3 Cosa aspettarsi come speedup

- **Autovec vs Baseline:** $1.5\text{--}3\times$ (dipende da quanto GCC riesce a vettorizzare la moltiplicazione 64-bit)
- **AVX2 vs Baseline:** $2\text{--}4\times$ (limitato dalla memoria, non dal compute)
- **CUDA kernel-only vs CPU:** ordini di grandezza più veloce
- **CUDA end-to-end vs CPU:** potrebbe essere **più lento** per N piccoli a causa del transfer overhead

7 Checklist per il Report Finale (max 4 pagine)

1. Design choices

- ☐ Motivare la scelta del Fibonacci/Multiply-Shift hashing
- ☐ Motivare P come potenza di 2 (shift vs modulo)
- ☐ Discutere la costante A (golden ratio, universalità)

2. Evidence auto-vectorization GCC

- ☐ Flag usati per abilitare/disabilitare
- ☐ Estratto del report (`-fopt-info-vec-*`)
- ☐ Interpretazione: cosa ha vettorizzato, cosa no, perché

3. Strategia intrinsics

- ☐ Spiegare la decomposizione della mul 64-bit
- ☐ Spiegare il packing 64 $\rightarrow$ 32 bit
- ☐ Gestione del tail loop

4. Risultati

- ☐ Tabella: N , P , tempo, throughput, speedup per ogni versione
- ☐ Grafico: speedup al variare di N e/o P
- ☐ Discussione bottleneck (compute vs memory vs overhead)
- ☐ Analisi distribuzione per diversi P e `key_space`

5. CUDA (se implementato)

- ☐ Verifica correttezza (checksum match)
- ☐ Breakdown: H $\rightarrow$ D, kernel, D $\rightarrow$ H
- ☐ Confronto kernel-only vs end-to-end vs CPU

8 Consigli e Pitfall

Attenzione

Pitfall comuni:

1. **Overflow nella moltiplicazione AVX2:** assicurati di gestire correttamente tutti e 3 i prodotti parziali. Testa con chiavi grandi ($> 2^{32}$).
2. **Shift amount:** con $P = 2$, lo shift è 63. Con $P = 1024$, lo shift è 54. Verifica che il codice funzioni per tutti i valori.
3. **Allineamento:** `_mm256_load_si256` richiede allineamento a 32 byte. Usa `aligned_alloc` o `_mm256_loadu_si256` (unaligned, leggermente più lento).
4. **Benchmarking:** non misurare la generazione delle chiavi! Solo il kernel di mapping.
5. **rsync permissions:** mai copiare in `~/` con `-a`. Sempre in una sottodirectory.

Suggerimento

Best practices:

1. Inizia con N piccolo per debuggare, poi scala.
2. Usa `__builtin_ctz(P)` per calcolare $\log_2(P)$ in modo sicuro.
3. Per il report, usa `gnuplot` o `matplotlib` per i grafici.
4. Compila con `-march=native` su node09 per abilitare tutte le estensioni supportate.
5. Verifica che il tuo codice compili anche con `-Wall -Wextra` senza warning.
6. La costante `HASH_A` è la stessa in tutte le implementazioni $\Rightarrow$ garantisce output identici.

9 Riferimenti

- [1] P. Ferragina, *Pearls of Algorithm Engineering*, Cambridge University Press, 2023. Cap. 8: The Dictionary Problem, §8.2–8.3 (Hash Tables, Universal Hashing, Multiply-Add-Shift).
- [2] D. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed., 1998. §6.4 (Hashing, Fibonacci hashing).
- [3] Lezioni SPM 2025/26:
 - Lez. 7&8: SIMD programming on CPUs (intrinsics, auto-vectorization)
 - Lez. 9: SIMT on GPUs (CUDA)
 - Lez. 10: Metrics and Laws (speedup, efficiency, Amdahl)
 - Lez. 5&6: Shared Memory (cache, roofline model)
 - Lez. 4: SLURM
- [4] M. Dietzfelbinger et al., “A Reliable Randomized Algorithm for the Closest-Pair Problem”, *J. Algorithms*, 1997. (Multiply-shift universality proof)
- [5] Intel, *Intrinsics Guide*: <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>